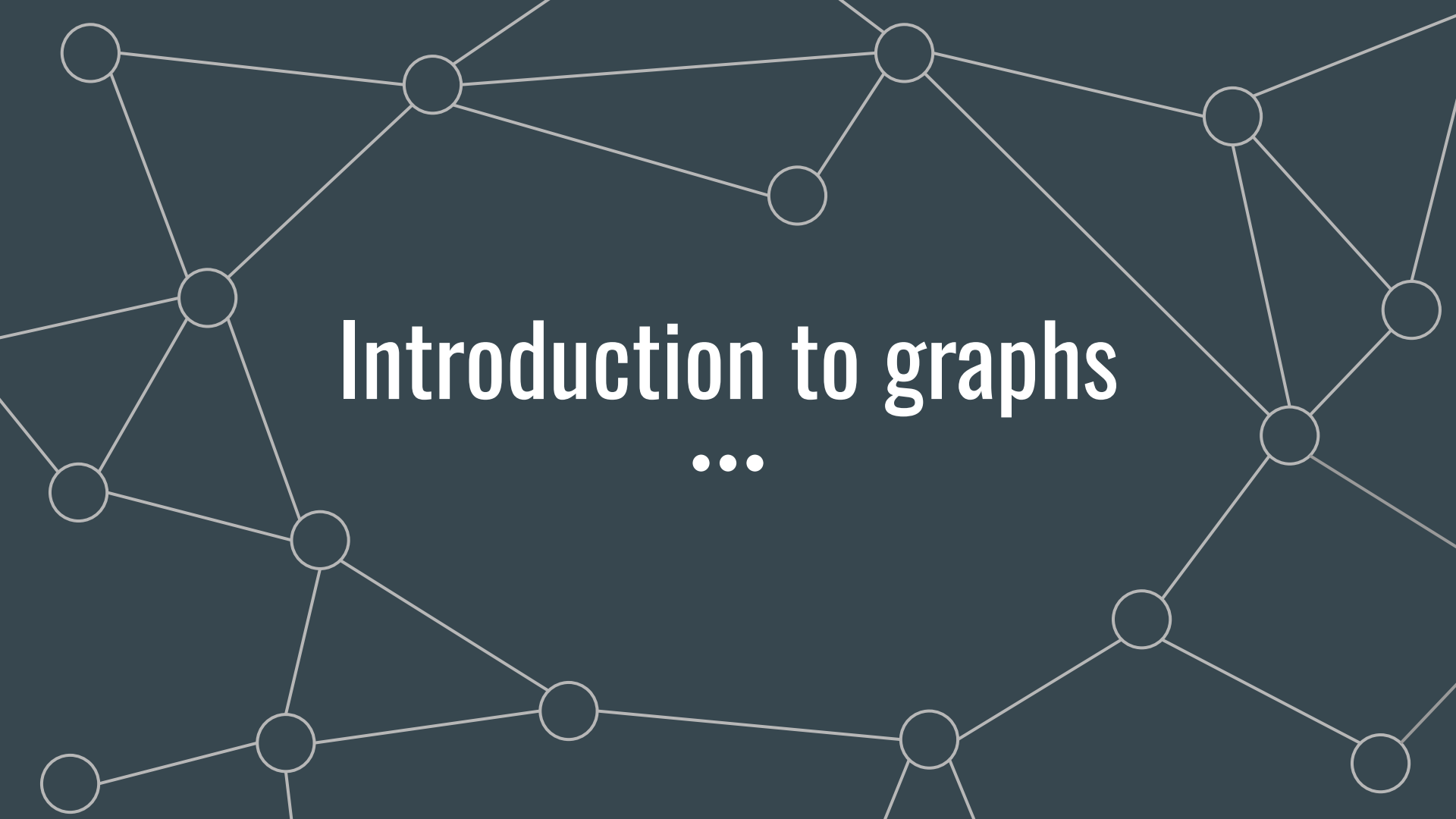


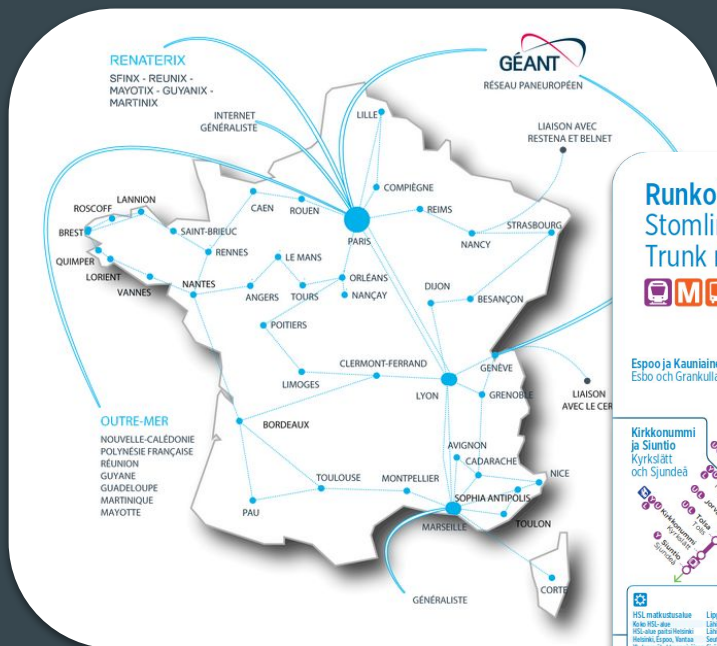
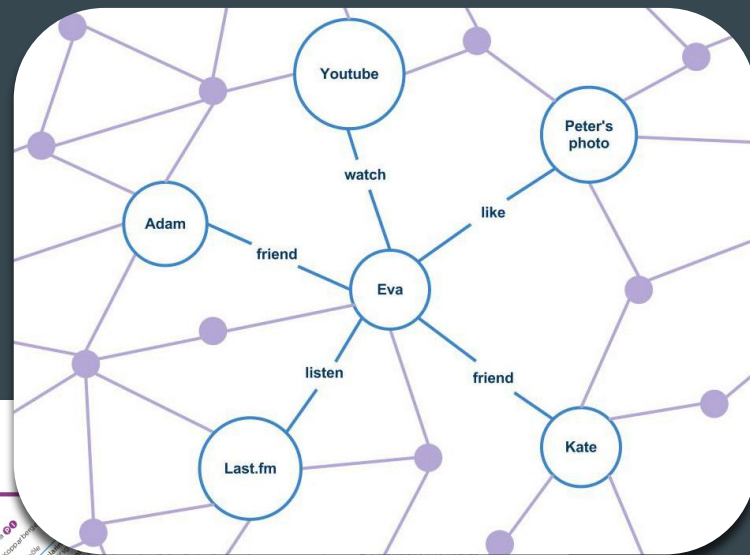


Introduction to graphs

...

A world of graphs

*Social
interactions*



Renater network



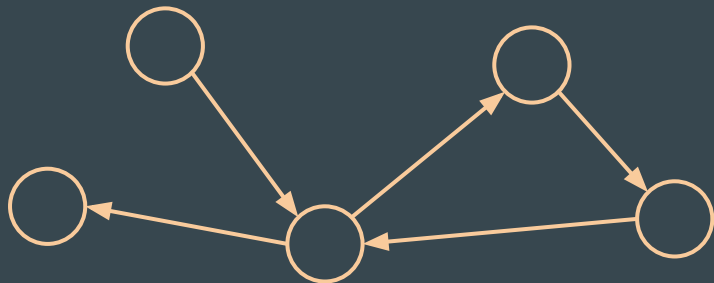
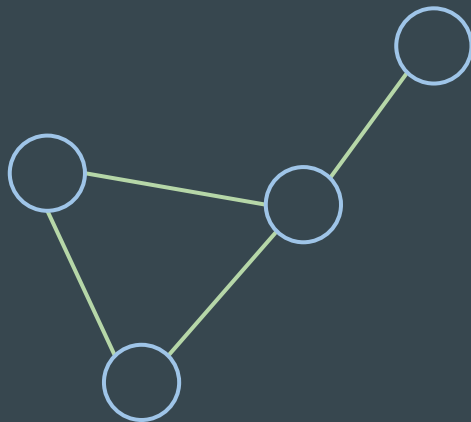
*Helsinki
trains
map*

So what's a graph?

It's a tuple (V, E) , where

- V is a **set of vertices**
- E is a **set of edges**

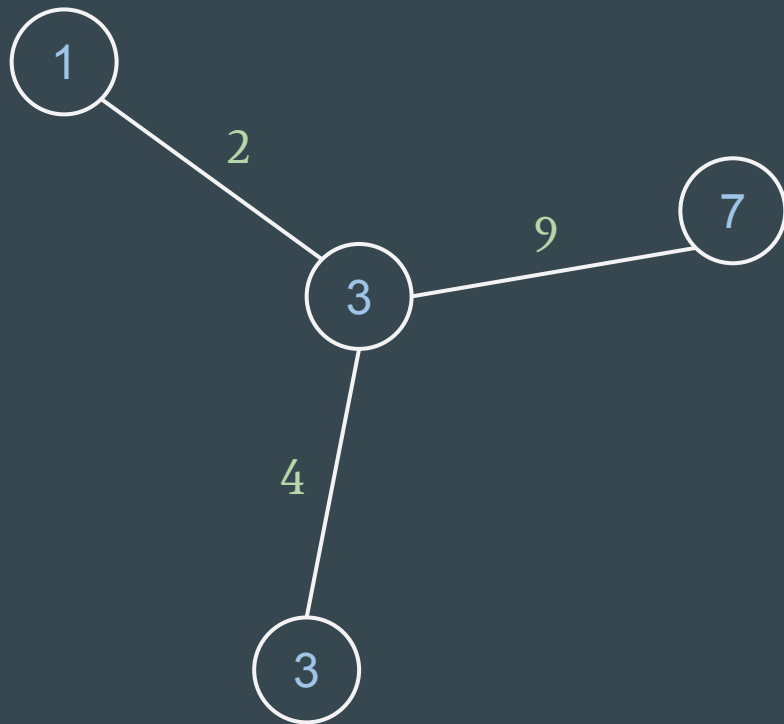
If the edges have an orientation, the graph is **directed**, otherwise it is **undirected**



So what's a graph?

Some graphs contain more information such as:

- values on vertices
- values on edges



How to represent a graph?

Solution #1 : edge list

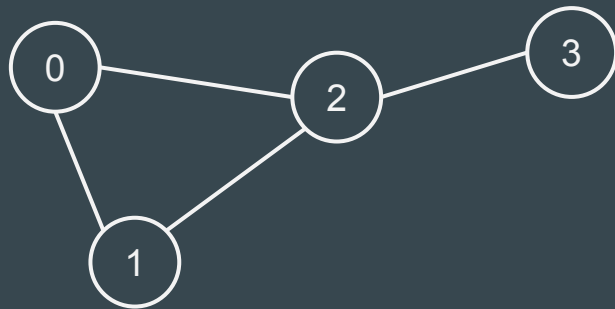
Let's label the vertices from 0 to $n-1$

We need to find a data structure for the edges...

What about a list of couples?

Check if i and j are neighbors	$O(E)$
Find all neighbors of i	$O(E)$

→ awfully long for such simple operations



$[(0,1), (0,2), (1,2), (2,3)]$

How to represent a graph?

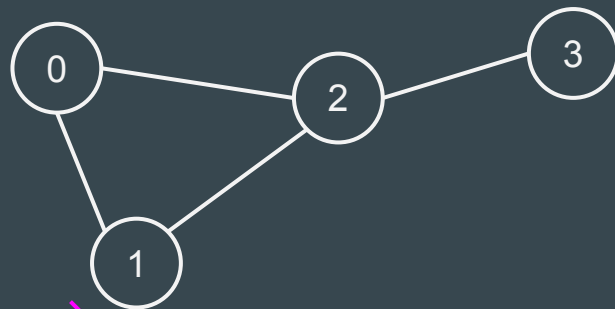
Solution #2 : adjacency matrix

Another idea is to store the relations as a matrix

→ boolean, or the value on the edge if the graph has some (and a dead value like None if there is no edge between two nodes)

Check if i and j are neighbors	$O(1)$
Find all neighbors of i	$O(n)$

→ but always takes $O(n^2)$ in memory
(not suitable for big graphs)



0	1	1	0
1	0	1	0
1	1	0	1
0	0	1	0

Symmetry for a directed graph

How to represent a graph?

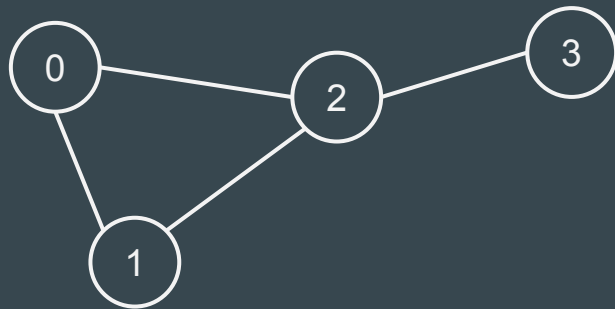
Solution #3 : Adjacency list

→ store all neighbors of each node

(there is a variant with sets, they take more memory but allow quicker adjacency check)

Check if i and j are neighbors	$O(E / n)$ avg
Find all neighbors of i	$O(E / n)$ avg

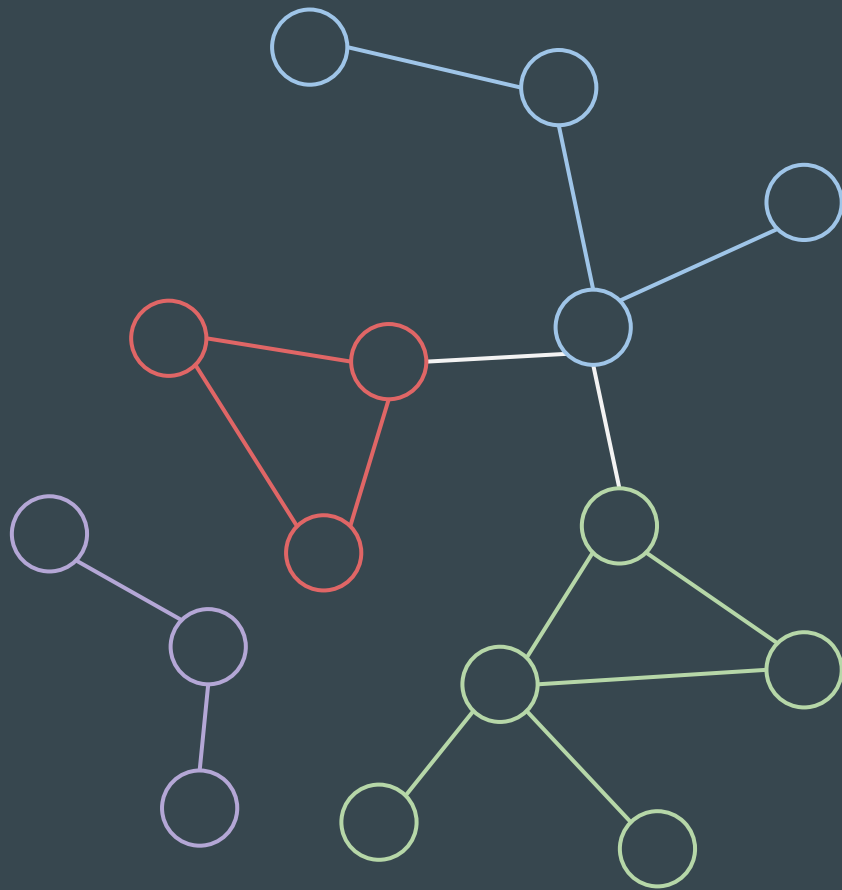
→ this representation is very often preferred as it is compact and efficient



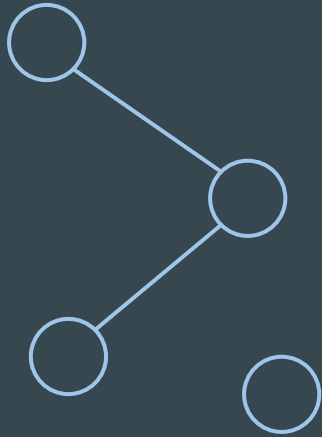
```
{0: [1, 2],  
 1: [0, 2],  
 2: [0, 1, 3],  
 3: [2]}
```

Just a few terms

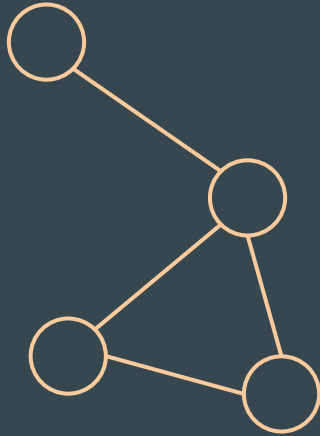
- a **path** is a sequence of 2-by-2 adjacent nodes and edges
- a **simple path** is a path that has no repeated vertices and edges
- a **cycle** is a path starting and ending at the same node
- a **connected component** is a subgraph in which there exists a path between any two nodes but not with any external node



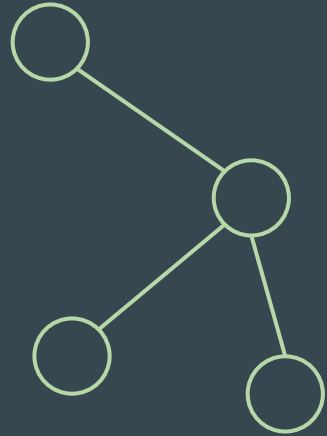
Special graphs (undirected)



acyclic graph



connected graph



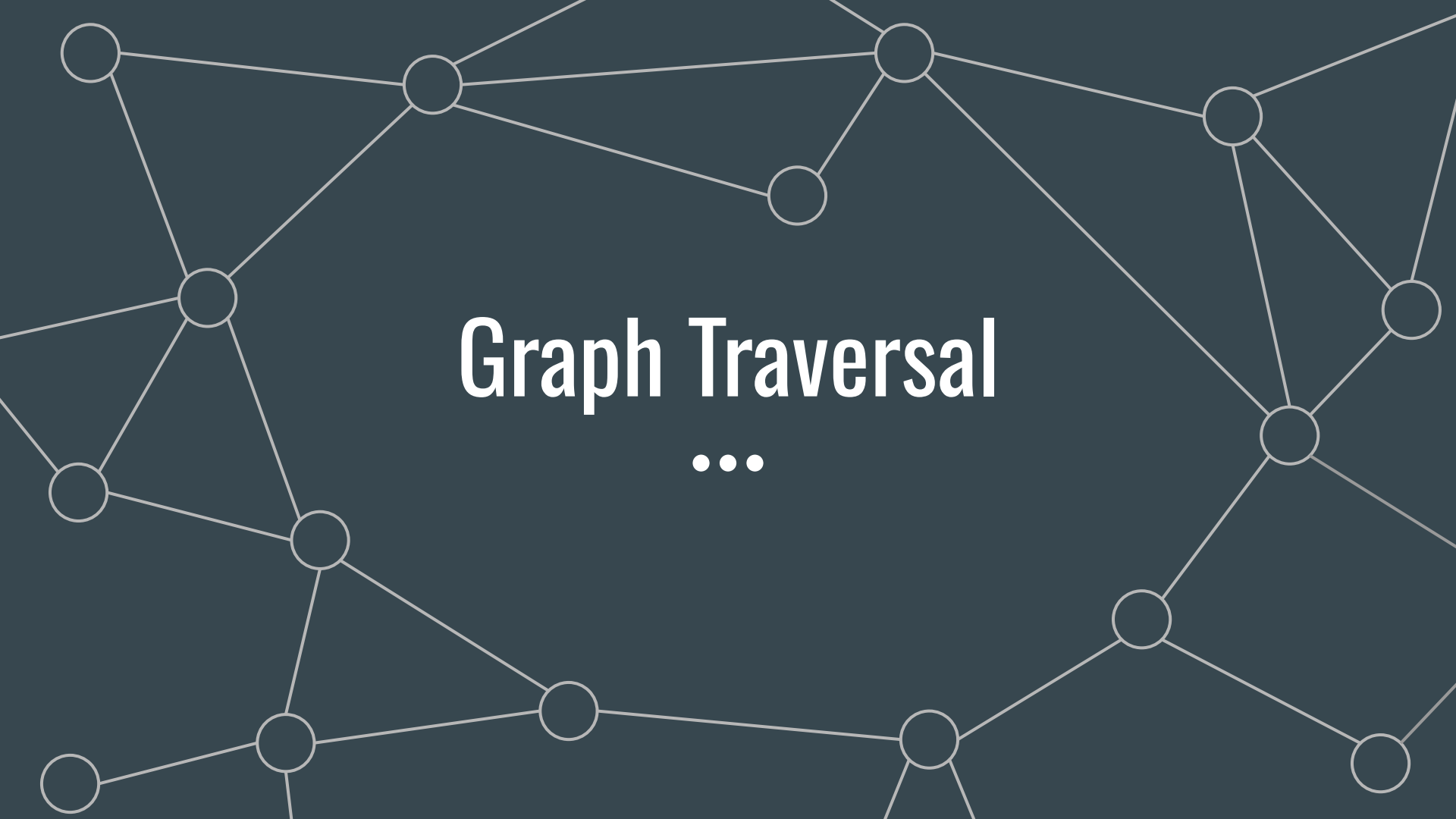
tree = connected + acyclic

Note on directed graphs

- their matrix isn't symmetrical
- being acyclic doesn't mean that the undirected counterpart is acyclic too
- there are two notions of connection for directed graphs:
 - weakly connected = the corresponding undirected graph is connected
 - strongly connected = there is a path in each direction for each pair of vertices

Some properties of trees...

- A tree has $N-1$ edges for N vertices
- A tree is a connected & acyclic graph
- Remove any edge from the tree and it disconnects the graph
- In the general case vertex can have any degree
 - Some trees are more specific : binary trees, k -trees



Graph Traversal

...

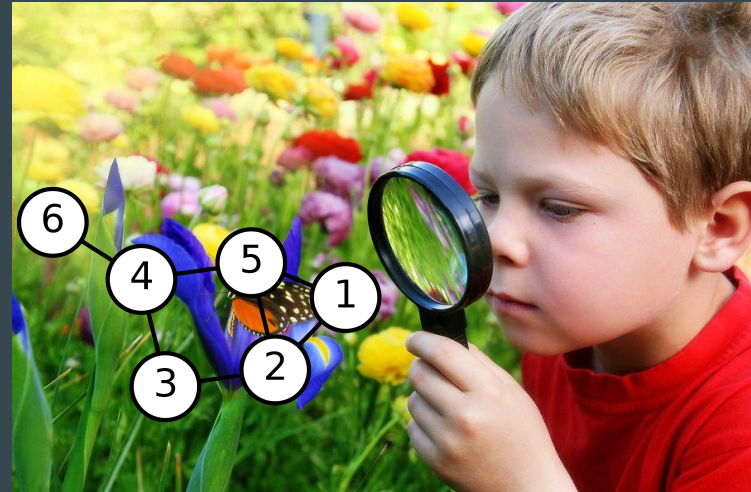
Graph traversal

What is it ?

-> A method to explore a graph from a given node (every node accessible)

Purpose ?

- Quickly explore the graph (useful for some problems).
- Foundation for more advanced algorithms.



Big notions used



- Visited/unvisited:

To explore the graph optimally (a node is explored only once) we need to keep track of which node has been already visited

- Main data structure:

The data structure used to keep track on which node will be visited next.

- Additional data structure:

May be used to keep track of various informations depending on the context

Two algorithm for the price of one:

BFS

Breadth-first search

-

Explore first the nearest nodes from the
current position

FIFO - Queue

DFS

Depth-first search

-

Go as deep as you can and backtrack when
you're in a dead end

LIFO - Stack

A small demonstration

BFS

DFS

BFS

```
BFS(G, start):  
    let q be a queue  
    add start to the queue  
    set every node of G to unvisited  
    set start to visited  
  
    while q is not empty:  
        current is the next node from q  
  
        for every node  $a_i$  adjacent to  
        current:  
            if  $a_i$  is visited: skip  
            set  $a_i$  to visited  
            put  $a_i$  in q
```

Complexity : $O(E+V)$

DFS

```
DFS(G, current):  
    mark node current as visited  
    for every node nxt adjacent to current:  
        if nxt is not visited:  
            DFS(G, nxt)
```

Complexity : $O(E+V)$

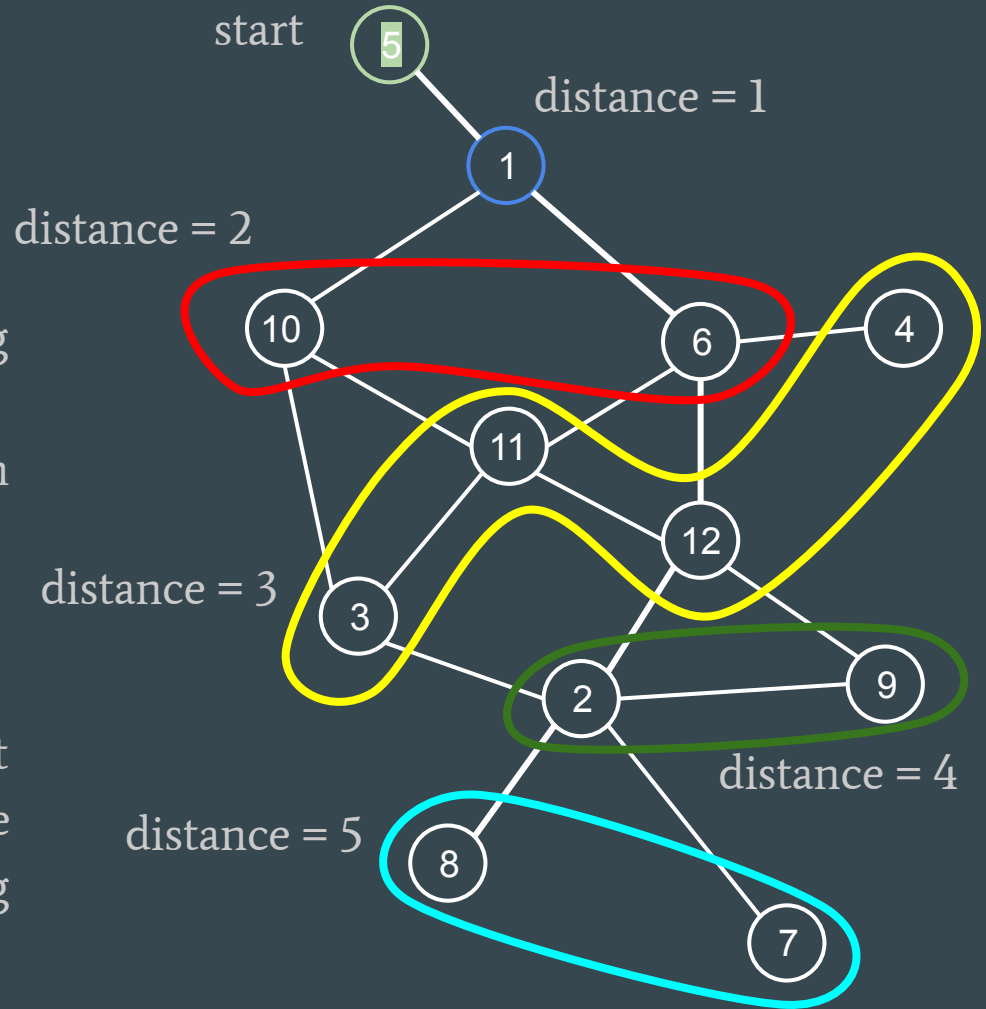
Why two variants ?

BFS:

- Processes node in increasing distance from starting node
- => Find the shortest path from starting node to every node.

Queue: [5, 1, 10, 6, 11, 3, 4, 12, 2, 9, 8, 7]

- BFS is beneficial if we know target node is close to starting node because we can stop searching there.



Why two variants ?

DFS:

- Very easy to code (5 lines for the recursive version)
- Memory efficient (no need for extra lists)
- Allows topography sorting of graph

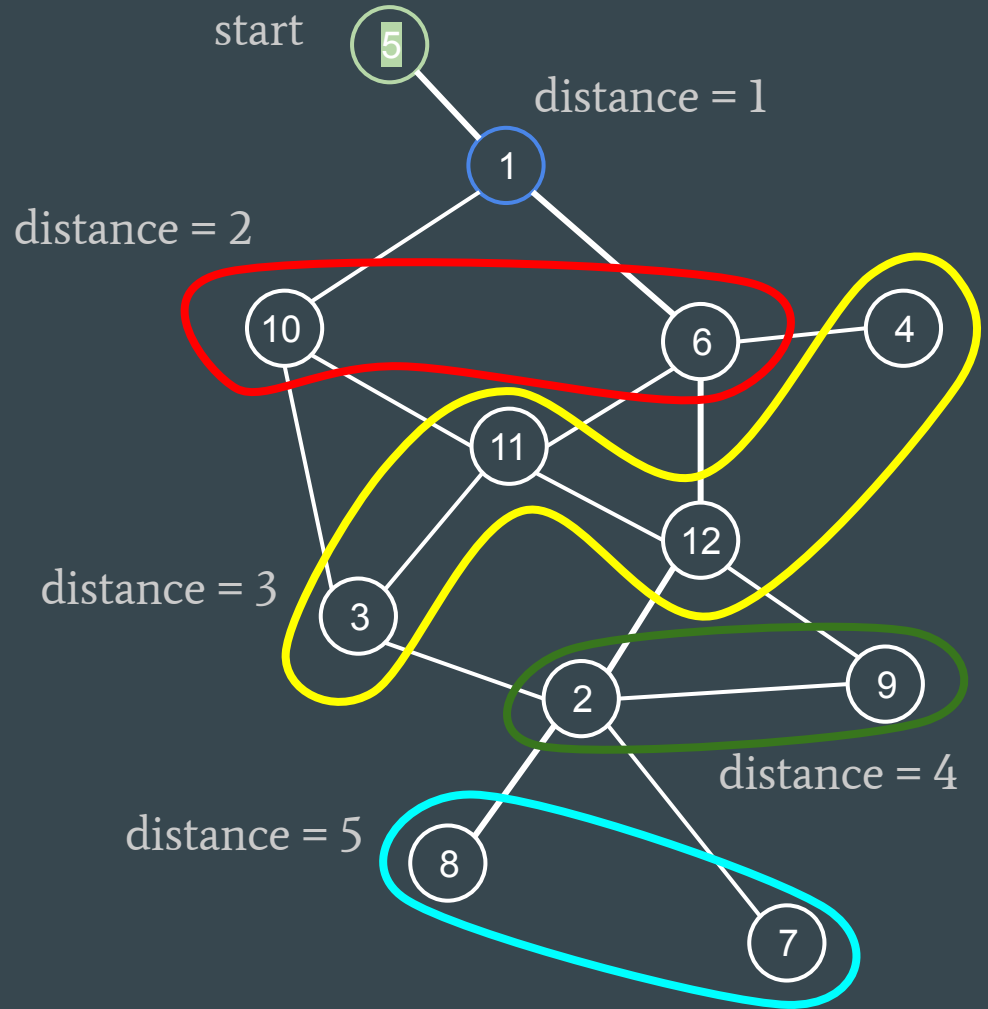


Shortest path (Dijkstra)

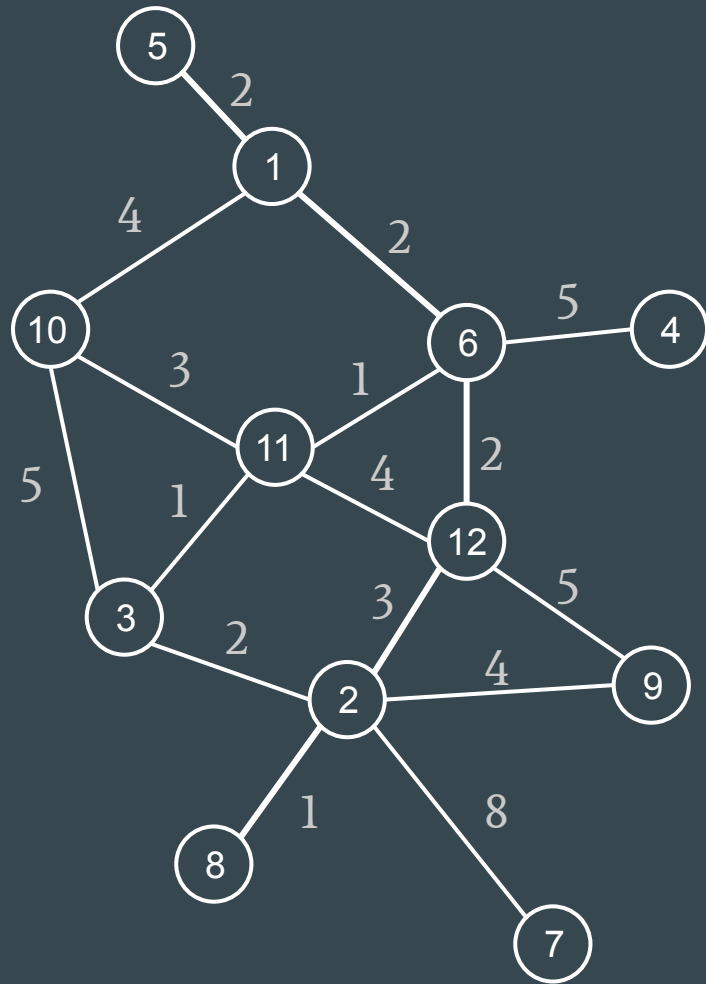
...

BFS processes nodes in the order of increasing distance from starting node, hence can find shortest path to each node.

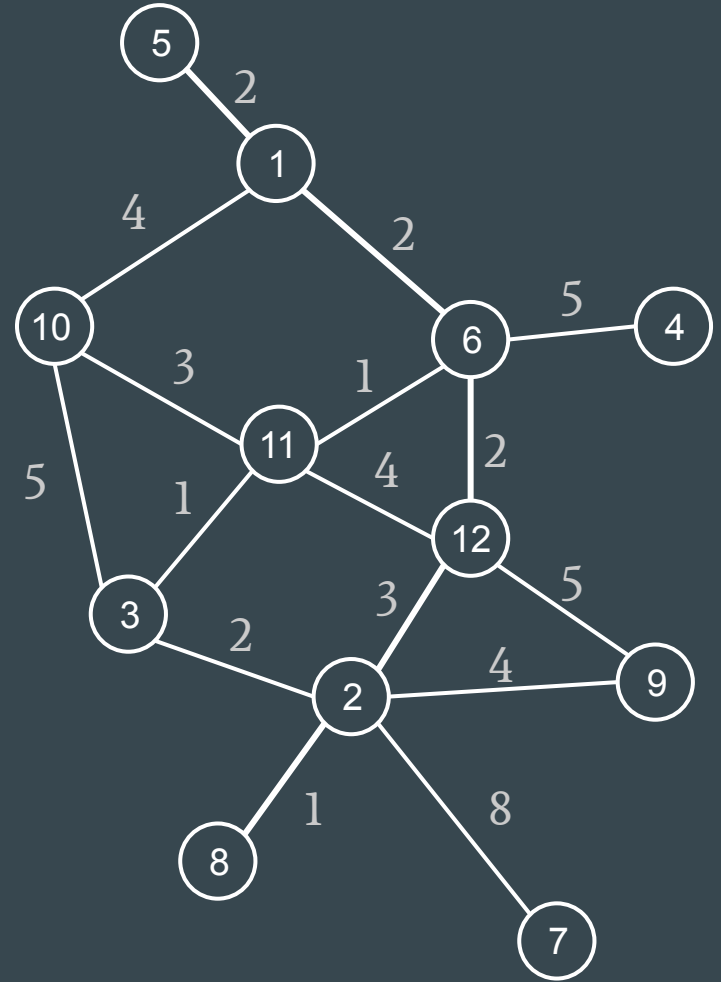
Queue: [5, 1, 10, 6, 11, 3, 4, 12, 2, 9, 8, 7]



But what if each edges had a weight?

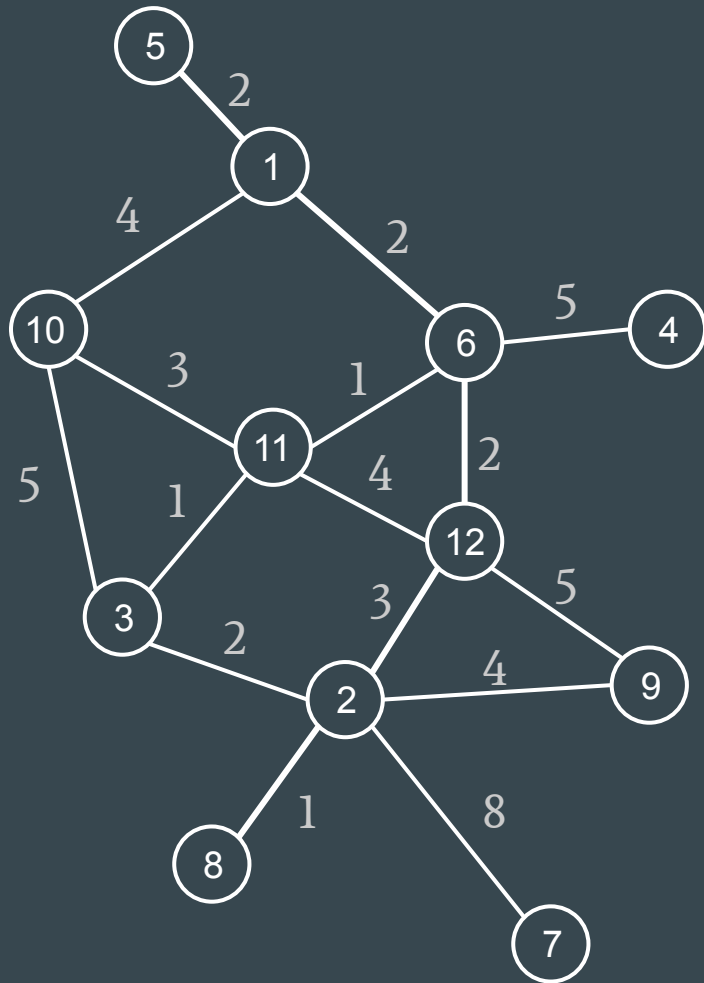


But what if each edges had a weight?
=> Breaks BFS logic, doesn't visit
nodes in increasing order of distance



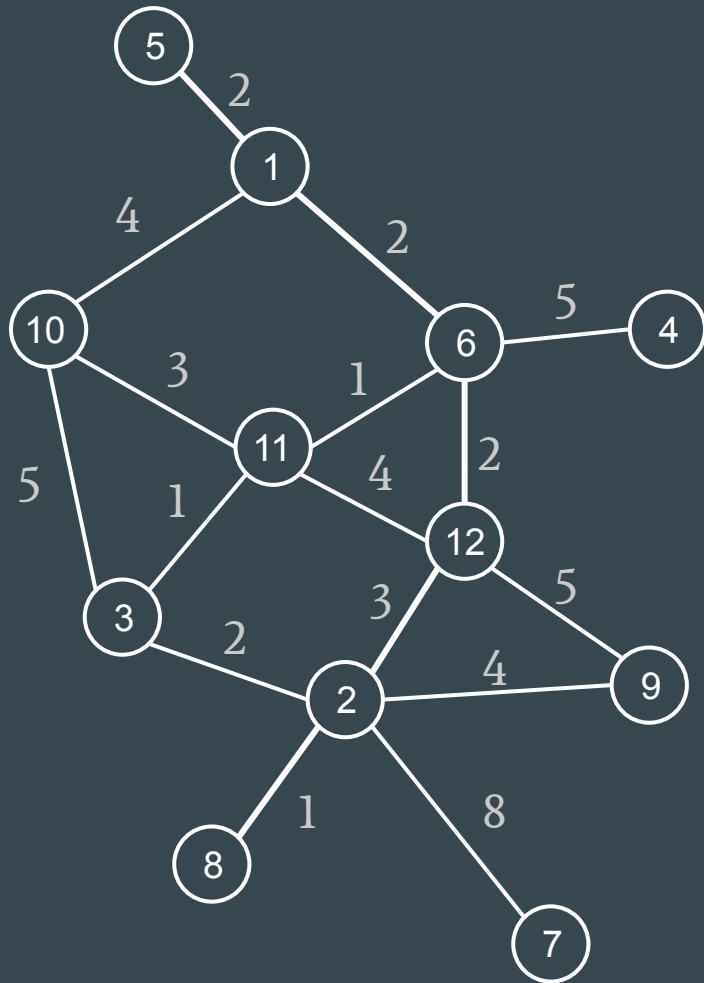
=> We think about modifying BFS to maintain visiting nodes in increasing distance from starting node.

```
BFS(G, start):  
    let q be a queue list  
    add start to q  
    set start to visited  
    let dist be a list representing distance to  
starting node  
    initialize dist to infinite  
    dist[start] = 0  
    while q is not empty:  
        current is the next node from q  
        current is the node shortest from starting  
node in q  
        if current is visited: continue  
        set current to visited  
  
        for every node  $a_i$  adjacent to current with  
edge weight w:  
            if dist[ $a_i$ ] > dist[current] + w:  
                dist[ $a_i$ ] = dist[current] + w  
                put  $a_i$  in q
```



This is actually the idea behind Dijkstra!

```
Dijkstra(G, start):  
    let q be a list  
    add start to q  
    set start to visited  
    let dist be a list representing distance to  
starting node  
    initialize dist to infinite  
    dist[start] = 0  
  
    while q is not empty:  
        current is the node shortest from starting  
node in q  
        if current is visited: continue  
        set current to visited  
  
        for every node  $a_i$  adjacent to current with  
edge weight w:  
            if dist[ $a_i$ ] > dist[current] + w:  
                dist[ $a_i$ ] = dist[current] + w  
                put  $a_i$  in q
```



```

Dijkstra( $G$ ,  $start$ ):
    let  $q$  be a list
    add  $start$  to  $q$ 
    set  $start$  to visited
    let  $dist$  be a list representing distance to
starting node
    initialize  $dist$  to infinite
     $dist[start] = 0$ 

    while  $q$  is not empty:
        current is the node shortest from starting
node in  $q$ 
        if  $current$  is visited: continue
        set  $current$  to visited

        for every node  $a_i$  adjacent to  $current$  with
edge weight  $w$ :
            if  $dist[a_i] > dist[current] + w$ :
                 $dist[a_i] = dist[current] + w$ 
                put  $a_i$  in  $q$ 

```

Time complexity:
 $O((V + E)^2)$

$O(V+E)$
have to traverse through
all elements in q

It could be better optimized !

Introducing heapq in Python:

2 operations:

`heapq.heappop(q):`

Return and remove minimum value from heap `q`

Time complexity: $O(\log n)$

`heapq.heappush(q, v):`

Add value `v` to heap `q`

Time complexity: $O(\log n)$

It could be better optimized !

Introducing heapq in Python:

2 operations:

`heapq.heappop(q):`

Return and remove minimum value from heap `q`

Time complexity: $O(\log n)$

`heapq.heappush(q, v):`

Add value `v` to heap `q`

Time complexity: $O(\log n)$

Now, use it to improve Dijkstra complexity

```
import heapq
```

```
Dijkstra(G, start):
```

```
    let q be a list
```

```
    heapq.heappush(q, (0, start))
```

```
    set start to visited
```

```
    let dist be a list representing distance to starting node
```

```
    initialize dist to infinite
```

```
    dist[start] = 0
```

```
while q is not empty:
```

```
    D, current = heapq.heappop(q)
```

```
    if current is visited: continue
```

```
    set current to visited
```

```
    for every node  $a_i$  adjacent to current with edge  
    weight w:
```

```
        if dist[ $a_i$ ] > dist[current] + w:
```

```
            dist[ $a_i$ ] = dist[current] + w
```

```
            heapq.heappush(q, (dist[ $a_i$ ],  $a_i$ ))
```

Time complexity:

$O((V + E) \log_2 (V+E))$

$O(\log_2 (V+E))$



Slides: Louis Sugy for INSAIgo
revised in 2020 by Louis Gombert
revised in 2025 by Ngoc Ha Dao

<i>Helsinki trains map:</i>	HSL
<i>The Internet in 2015:</i>	The Opte Project
<i>Renater network:</i>	Renater

Credits

More resources:

- DFS : https://en.wikipedia.org/wiki/Depth-first_search
- BFS : https://en.wikipedia.org/wiki/Breadth-first_search
- Cours de 3IF : https://moodle.insa-lyon.fr/pluginfile.php/436811/mod_resource/content/1/cours.pdf

Slides: Sebastien Goll for INSAIgo